An independent partial check of the three-dimensional Yukawa bcc-to-fcc threshold of Luo and Wei: the threshold reproduced three ways, seventeen Lean statements kernel-checked, and what remains open

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-27

doi:10.5281/zenodo.23708332 (not peer reviewed)

Zenodo DOI to be assigned on release (not peer reviewed)

AI-assistance disclosure. This preprint, its Lean code, its Python code and its claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction. The author is responsible for the content. No human has audited the Lean statements for adequacy, and no human referee has checked the hand arguments. An independent AI audit (`AUDIT.md`, verdict ACCEPT_WITH_CORRECTIONS, no blocking item) rebuilt the five main Lean modules from source into a private olean directory and replayed them with `leanchecker` (rc 0 on all five, `data/leanchecker_replay_audit.json`; its oleans are byte-identical to the producer's); this revision applies its corrections (`REVISION.md`). **No human audit and no statement lock exist.**

Credit. The result discussed here is due to Senping Luo and Juncheng Wei, *On exact solutions to three-dimensional Yukawa lattice model and BCC to FCC phase transitions*, arXiv:2610.31456v1 [math.AP], 22 October 2026 [1]. They determine the global minimisers of the Yukawa lattice energy at every screening and prove a single bcc-to-fcc transition at $\kappa_* \approx 1.7179218684699$. The credit for that result belongs to them. This note is a *check* of part of their proof, not a refutation: **every printed number and inequality that we could check holds, and we found no discrepancy.**

Abstract

For unit-density point charges that repel through the screened Coulomb (Yukawa) potential $e^{-\kappa r}/r$, Luo and Wei prove that the body-centred cubic (BCC) lattice has the lowest energy among all three-dimensional lattices for $\kappa < \kappa_*$, the face-centred cubic (FCC) lattice for $\kappa > \kappa_*$, and that $\kappa_* \approx 1.7179218684699$. Their proof leans on finite, computer-assisted inequalities. We checked part of them independently. **We found no discrepancy; every printed number and inequality that we checked holds.** Tier split. [A] (Lean 4.34.1, pinned Mathlib; axioms within {`propext`, `Classical.choice`, `Quot.sound`}; `leanchecker` replay rc 0 on `Basic`, `Shells`, `Chains`, `LemmaA5`, `Lemma61`, run by the producer and again by the independent AI audit on privately rebuilt, byte-identical oleans; all 138 declarations of the five modules print only these axioms; no human audit; no statement lock): *seventeen Lean theorems* (ten substantive; seven are one-line rational facts or bridge lemmas, and one of them, `lemma61_prop41_bracket`, is a for-all-x inequality, not a finite statement): the FCC shell multiplicities $N_1, \ldots, N_{12}$ and the BCC multiplicities M_ℓ at the 17 nonzero indices $\ell \le 48$, proved equal to the count over all of $\mathbb{Z}^3$; Lemma A.5 (A.6), (A.7) and (A.8) as real-number statements over `Real.pi` and `Real.exp`;

the constants of the proof of Lemma 6.1; the tail rational of the proof of Lemma A.6; and the rational step of Lemma A.1. Several Lean statements are weaker than the printed ones (Table 2). [L]: the paper's own statements, taken as stated. [C]: Arb ball arithmetic (Lemma A.6 for $G = 2(E_B - E_F)$ at $\kappa \in \{1.717921868469903, 1.717921868469904\}$ and at neighbouring controls; the Lemma A.5 minimum slacks 7.52×10^{-4} and 7.65×10^{-5}; Lemmas A.1 and A.3) and *three independent reproductions of* κ_* (Ewald sums in Arb at 320 bits; a direct real-space Arb sum; mpmath quadrature of the paper's integral equation (1.4) at 40 digits): (E) and (Q) agree to all 25 printed digits and the interval of (D), of width 4×10^{-24}, contains both. The audit added a fourth, structurally different value (classical Ewald split with a free splitting parameter, mpmath, 70 digits, independent of that parameter), $\kappa_* = 1.71792186846990386425087416660628171879468538$, and an independent ball-arithmetic run (python-flint 0.9.0) that reproduces both signs of Lemma A.6. (Our IEEE-double direct sum reaches 11 digits; double precision cannot resolve the 10^{-15}-wide enclosure (A.9), since rounding at 10^{-16} in sums of order one moves κ_* by about 10^{-12}.) **Not done:** the assembly of Lemma 6.1 (theta functions as lattice sums, Poisson summation for $D(1) = 0$, $N_n < 30n$ for all n); Lemma A.4 (5 828 rectangles: the partition data is not public and arXiv lists no ancillary files; we intend to ask the authors for it) and Lemma A.2; and Lemma A.6 in Lean (the pinned Mathlib has no `erfc`). Every claim in the text carries a tier tag and a ledger identifier (`YBC-·-nnnn`).

What this is NOT. This concerns the *stated mathematical model* of point charges with screened Coulomb repulsion on a lattice at zero temperature. It makes **no claim about any real material**, plasma, colloid or crystal. It is not a verification of Theorem 1.1 of Luo and Wei, and not a refutation of it.

Markers and ledger. [A]: kernel-checked Lean, axioms within the allowlist, controls caught. [L]: a cited source, taken as stated. [C]: executed code with an argument written down but not machine-checked in Lean (here: interval/ball arithmetic and quadrature). [X]: floating point, supporting nothing by itself. A [A] tag attaches to the Lean statement, never to its physical gloss. Identifiers `YBC-·-nnnn` refer to `ledger/ledger.json` (Section 8).

1 Setting and scope

This is item M3 of a computational programme (Proposal 4, preregistered 2026-10-27 in `results/proposal4/2026-10-27/preregistration.json`; the file was not edited). The preregistered aims were: (M3a) kernel-check Lemma A.5 and Lemma 6.1; (M3b) kernel-check the two signed bounds of Lemma A.6; (M3c) re-run Lemma A.4 (5 828 rectangles) and Lemma A.2 in ball arithmetic. The kill rule said: if rational exp enclosures at the 10^{-20} margin were too slow in the kernel, kernel-check Lemma 6.1 only and report A.6 as Tier C. The outcome is the preregistered *reduced* one, with two differences set out in Section 9: A.6 went to Tier C because Mathlib lacks `erfc`, not because the kernel was too slow, and Lemma 6.1 itself is *not* in Lean, only its finite inputs. Overall status: **PARTIAL**. The prior assigned before any attempt was 0.45 for the headline.

We never write "Luo and Wei is verified". We say which finite statements were checked, at which tier, and which parts were not touched. Had a discrepancy appeared, the protocol was to re-check it a second time and tell the authors before saying anything public.

2 A step-by-step account for non-specialists

Each step ends with the Lean statement that corresponds to it, when there is one. Lean statements are transcribed in ASCII; the sources are in `data/lean/`.

Step 1: the question. [L] Put point charges at the points of a lattice (a regular 3D grid generated by three vectors) with one point per unit volume. If the charges repel as $1/r$ (Coulomb), the lowest-energy lattice is BCC: a cube with a point at each corner and one at the centre. If the repulsion is *screened*, $e^{-\kappa r}/r$ (Yukawa), with screening $\kappa > 0$, strong screening makes the interaction short-ranged, and nearest-neighbour geometry wins: FCC, a cube with points at the corners and face centres, packs the nearest neighbours better. The energy per particle is $E_\kappa(L) = \frac{1}{2}\sum_{v\in L\setminus\{0\}} e^{-\kappa|v|}/|v|$. Luo and Wei prove that among *all* lattices of unit covolume the minimiser is BCC for $\kappa < \kappa_*$ and FCC for $\kappa > \kappa_*$.

Step 2: what a "certified threshold" means. [L] The threshold κ_* is the unique κ with $E_\kappa(L_{\mathrm{B}}) = E_\kappa(L_{\mathrm{F}})$. A *certified* enclosure of κ_* means this: evaluate $G(\kappa) = 2(E_\kappa(L_{\mathrm{B}}) - E_\kappa(L_{\mathrm{F}}))$ at two rational values $\kappa_1 < \kappa_2$ and show rigorously that $G(\kappa_1) < 0 < G(\kappa_2)$. Because G has a single sign change, $\kappa_* \in (\kappa_1, \kappa_2)$. Lemma A.6 does this with $\kappa_1 = 1717921868469903/10^{15}$ and $\kappa_2 = 1717921868469904/10^{15}$, claiming $G(\kappa_1) < -10^{-19}$ and $G(\kappa_2) > 10^{-20}$. The numbers are small because $G'(\kappa_*) \approx 1.57 \times 10^{-4}$: one unit in the fifteenth decimal moves G by about 1.6×10^{-19}. [C]

Step 3: shell multiplicities. [L] The FCC lattice L_{F} and its dual BCC lattice $L_{\mathrm{B}} = L_{\mathrm{F}}^*$ have squared lengths $2^{1/3}n$ (with N_n vectors) and $2^{-4/3}\ell$ (with M_ℓ vectors). Concretely $N_n = \#\{m \in \mathbb{Z}^3 : |m|^2 = 2n\}$ and $M_\ell = \#\{m \in \mathbb{Z}^3 : |m|^2 = \ell,\ m_1 \equiv m_2 \equiv m_3 \bmod 2\}$. Everything below uses the printed tables. We recounted them in the kernel, over a finite box, *and* proved that the box count equals the count over all of $\mathbb{Z}^3$:

```
theorem Nc_eq_ncard (n : N) (hn : 2*n < 36) :
    Nc n = Set.ncard {m : Z x Z x Z | normSq m = 2*n}
theorem Mc_eq_ncard (l : N) (hl : l < 49) :
    Mc l = Set.ncard {m : Z x Z x Z | normSq m = l /\ sameParity m}
theorem fcc_multiplicities :
    (List.range 12).map (fun i => Nc (i+1)) = [12,6,24,12,24,8,48,6,36,24,24,24]
theorem bcc_multiplicities :  -- 17 nonzero M_l, l <= 48
    filter (Mc l /= 0) [1..48] = [3,4,8,11,12,16,19,20,24,27,32,35,36,40,43,44,48] /\
    map Mc [3,4,8,...,48] = [8,6,12,24,8,6,24,24,24,32,12,48,30,24,24,24,8]
```

[A] (`YBC-A-0001`, `YBC-A-0002`). The box sizes (radius 5 and 6) are justified by the first lemma: any vector of squared length below $(R+1)^2$ has all coordinates in $[-R, R]$.

Step 4: a finite inequality becomes a statement about real numbers. [L] Lemma A.5 compares two sums of exponentials on a grid of $1000 + 2000$ cells. Here is (A.7): for every integer $0 \le j < 2000$, $\sum_{\ell\le 48} M_\ell e^{-\pi 2^{-4/3}\ell\,(11/10+(j+1)/2000)} - \sum_{n\le 12} N_n e^{-\pi 2^{1/3}n\,(11/10+j/2000)} - 10^{-18} > 10^{-4}$. In Lean, over the reals with `Real.pi`, `Real.exp`, $a_2 = 2^{1/3}$ and $b_2 = 2^{-4/3}$:

```
theorem lemmaA5_A7 (j : N) (hj : j < 2000) :
  sum l in Icc 1 48, (Mc l : R) * exp (-(pi*b2) * l * (11/10 + (j+1)/2000))
  - sum n in Icc 1 12, (Nc n : R) * exp (-(pi*a2) * n * (11/10 + j/2000))
  - 1/10^18 > 1/10000
```

[A] (`YBC-A-0004`); (A.6) is `lemmaA5_A6` for $j < 1000$ with margin $3/1000$ (`YBC-A-0003`), and (A.8) is `8 * exp((pi*a2 - 3*(pi*b2)) * (21/10)) > 25/2` (`lemmaA5_A8`, `YBC-A-0005`).

Step 5: how the kernel checks 3 000 exponential inequalities. [A] The kernel cannot evaluate e^x, so we reduce each cell to rational arithmetic. (i) π is bracketed by Mathlib's `Real.pi_gt_d20` and `pi_lt_d20`; $2^{1/3}$ and $2^{-4/3}$ are bracketed through their cubes. (ii) For a rational z, $e^z = (e^{z/8})^8$ and `Real.exp_bound` gives a rational enclosure from a Taylor polynomial. (iii) The values $e^{-\pi a_2(1+k/10^4)}$

and three similar sequences are supplied as lists of dyadic rationals (denominator 2^{52}) by an *untrusted* script; the kernel checks that each entry follows the previous one within a certified enclosure (`chainLo`, `chainHi`), and the chain is lifted by induction to the real sequence. Because shell sums are monotone in the exponential, one table entry per cell suffices. (iv) Each of the 3 000 cell inequalities is a rational comparison decided by `decide +kernel`, in blocks of 100 cells. An emitter script mirrors every kernel check in exact `Fraction` arithmetic before Lean runs; its slacks, 7.5214×10^{-4} for (A.6) and 7.6459×10^{-5} for (A.7), match the Arb values. The untrusted tables cannot make a false statement pass: a tampered entry fails (Section 6).

Step 6: the proof of Lemma 6.1, as far as it is finite. [L] Lemma 6.1 says $D(t) = \Theta(t, L_{\mathrm{B}}) - \Theta(t, L_{\mathrm{F}})$ satisfies $D(1) = 0$ and $D(t) > 0$ for $t > 1$, where $\Theta(t, L) = \sum_{v \in L} e^{-\pi t |v|^2}$. The printed proof uses Poisson summation (for $D(1) = 0$, since $L_{\mathrm{B}} = L_{\mathrm{F}}^*$), then, for $t \geq 1$, bounds the shells beyond $n = 13$ and $\ell = 49$ using $N_n < 30n$, $M_\ell < 30\ell$ and a few numerical constants. Those constants are [A] (`YBC-A-0006`), for example

```
theorem lemma61_exp_fcc : exp (-(pi*a2)) < 1/49
theorem lemma61_exp_bcc : exp (-(pi*b2)) < 3/10
theorem lemma61_tail_fcc (K : N) :
  30 * sum k in range K, (13+k) * (1/49)^(13+k) < 1/10^18
```

The statement for *every partial sum* is what we proved; passing to the infinite series is not written in Lean. The assembly is Section 7.

Step 7: Lemma A.6 and why it is in Ball arithmetic, not Lean. [C] E_κ is not a finite sum. The paper (eqs. 2.3, 2.4) rewrites it by an Ewald split into rapidly convergent sums over L_{F} and L_{B} (truncated at $n \leq 12$, $\ell \leq 48$) whose terms involve an incomplete integral $I(q, c) = \int_1^\infty t^{-1/2} e^{-\pi q t - c/t} dt$, a complementary error function in closed form. The pinned Mathlib has no `erfc`, hence no certified enclosure for I in Lean without building that theory. We therefore verify Lemma A.6 in Arb (Section 5). The rational tail, $60(1/49)^{13}/(1 - 1/49) + 60(3/10)^{49}/(7/10) < 7 \times 10^{-21}$ (exact value 6.5267×10^{-21}), is [A] (`lemmaA6_tail`, `YBC-A-0007`); so is the rational step $55995/10^6 < 7/125$ of Lemma A.1 (`lemmaA1_rational`).

3 Anatomy of the Lean development

The Lean modules are in `formal_cert/Programmes/YukawaBccFcc/` (copies in `data/lean/`). They import `Mathlib.Tactic` and `Mathlib.Algebra.BigOperators.Fin`; never `import Mathlib`. They were compiled through `scripts/certified_numerics/lean_pinned.py` (Lean 4.34.1, Mathlib `d13f23b7`) and not through `anse/formal/lean_runner.py`, a deviation from the repository's usual route that was inherited from earlier lanes; the equivalent checks were rerun by the independent audit (`AUDIT.md`).

The tables that the kernel does not trust. The dyadic tables for $e^{-\pi a_2(1+k/10^4)}$, $e^{-\pi b_2(1+k/10^4)}$ and the two grids with step 1/2000 are produced by a script. The kernel never accepts a table as given: each entry must lie between the lower and upper rational enclosures obtained from the previous one, and the first entry and the step constant are checked against certified Taylor enclosures. A wrong table entry makes the chain check evaluate to `false`, which is what the control in Section 6 shows.

Module	Content	compile (s)	leanchecker
`Basic`	box counts and their `ncard` equality; π, $2^{1/3}$, $2^{-4/3}$ brackets; `exp` enclosure; chain lifting; monotone shell polynomials; geometric tail lemma	17.8	rc 0
`Shells`	`decide +kernel` enumerations; `fcc_multiplicities`, `bcc_multiplicities`	657	rc 0
`Chains`	four untrusted dyadic tables (denominator 2^{52}), each step kernel-checked	162	rc 0
`LemmaA5`	3 000 cell checks in 30 blocks of 100; `lemmaA5_A6`, `lemmaA5_A7`	2 009	rc 0
`Lemma61`	(A.8), the Lemma 6.1 constants and tails, the A.6 tail, the A.1 rational step	23	rc 0
`Axioms`, `Statements`	`#print axioms` and `#check` of the exported theorems	15, 12	—
`Controls`, `Reject*`	planted and tampered controls (never imported)	12–32	—

Table 1: The development. Times are the producer's wall-clock on a shared core, from the compile logs. `leanchecker` replays were run by the producer and independently by the audit (rc 0 on all five; audit compile times 13.5, 119.9, 76.1, 959.6, 11.7 s and replay times 59.3, 168.1, 89.7, 1091.2, 55.7 s for `Basic`, `Shells`, `Chains`, `LemmaA5`, `Lemma61`, on two other cores).

The multiplicities. For reference, the values proved (Step 3) are

n	1	2	3	4	5	6	7	8	9	10	11	12
N_n	12	6	24	12	24	8	48	6	36	24	24	24

ℓ	3	4	8	11	12	16	19	20	24	27	32	35	36	40	43	44	48
M_ℓ	8	6	12	24	8	6	24	24	24	32	12	48	30	24	24	24	8

The sum $N_1 + \cdots + N_{12}$ and these entries are the values printed in the proof of Lemma A.5; we enumerated them independently (`code/m3_shells.py`) rather than reading them from the paper, and the kernel reproduced them.

How to read a Tier A line. `lemmaA5_A7` says: for the specific real numbers named, with the Mathlib constant π and the Mathlib exponential, the displayed inequality holds for every $j < 2000$. It does *not* say that the lemma is used correctly in the paper's proof, that the shell tables encode the right lattices (that is the gloss, which a person must compare with the paper's definitions), or anything about physics. The ledger flags this explicitly (`LEDGER_UNAUDITED_TIER_A`).

Lean statements that are weaker than the printed ones. The independent audit compared every exported statement with the printed one. All are non-vacuous; the following are weaker or linked to the paper only through a lemma. The theorems not listed (`fcc_multiplicities`, `bcc_multiplicities`, `lemma61_exp_fcc`, `lemma61_exp_bcc`, `lemma61_pb_lt`, `lemma61_a2_gt`) state exactly the printed tables and constants (at $t = 1$ for the exponentials). The Lean runs were not routed through `anse/formal/lean_runner.py` (see above); the equivalent checks (compile, `#print axioms`, `sorryAx` rejected, axiom whitelist) were rerun by the independent audit (`AUDIT.md`) on all 138 declarations of the five modules.

Lean theorem	What Lean states	What the paper states or uses
`lemma61_exp_fcc_21`	$e^{-\pi a^2 t} < 1/128$ at $t = 21/10$ only	the proof of Lemma 6.1 uses it for all $t \geq 21/10$ (trivially extendable by monotonicity)
`lemma61_prop41_bracket`	the rational bracket $12 + 30((1-x)^{-2} - 1) < 25/2$ for $0 \leq x \leq 1/128$	the bound $\Theta(t, L_{\mathrm{F}}) - 1 < x\,(12 + 30(\dots))$ (Prop. 4.1 with $N_n < 30n$) is not in Lean
`lemma61_tail_fcc`, `lemma61_tail_bcc`	every partial sum of the rational majorants $30\sum n\,49^{-n}$ and $\frac{75}{2}\sum \ell^2 (3/10)^{\ell}$	the shell sums $\sum N_n e^{-\pi a^2 n t}$ (needs $N_n < 30n$ for all n, $t \geq 1$) and the passage to the infinite series are not in Lean
`lemmaA5_A6`, `lemmaA5_A7`, `lemmaA5_A8`	sums over the *box counts* `Nc`, `Mc`	the shell multiplicities N_n, M_ℓ; equal to them only through `Nc_eq_ncard` ($2n < 36$, i.e. $n \leq 12$) and `Mc_eq_ncard` ($\ell < 49$, i.e. $\ell \leq 48$)
`lemmaA6_tail`	the printed rational $< 7 \times 10^{-21}$	faithful, but it does not certify that this rational bounds the Ewald tail (checked by hand by the audit, Tier C)
`lemmaA1_rational`	$55995/10^6 < 7/125$	faithful and trivial; the substantive parts of Lemma A.1 are Tier C

Table 2: Lean statements weaker than, or only linked to, the printed ones.

4 Results at a glance

Seventeen Lean theorems are kernel-checked, of which ten are substantive (the two multiplicity tables, `lemmaA5_A6/A7/A8`, `lemma61_exp_fcc`, `lemma61_exp_bcc`, `lemma61_exp_fcc_21` and the two partial-sum tails) and seven are bridge lemmas or one-line rational facts (`Nc_eq_ncard`, `Mc_eq_ncard`, `lemmaA1_rational`, `lemma61_pb_lt`, `lemma61_a2_gt`, `lemma61_prop41_bracket`, `lemmaA6_tail`). Note that `lemma61_prop41_bracket` is a for-all-x statement, not a finite one, and that the "17" here is unrelated to the 17 nonzero BCC multiplicity indices. The `#print axioms` output of each of the seventeen is exactly [`propext`, `Classical.choice`, `Quot.sound`] (`data/logs/lean_axioms.json`): `Nc_eq_ncard`, `Mc_eq_ncard`, `fcc_multiplicities`, `bcc_multiplicities`, `lemmaA5_A6`, `lemmaA5_A7`, `lemmaA5_A8`, `lemma61_exp_fcc`, `lemma61_exp_bcc`, `lemma61_pb_lt`, `lemma61_a2_gt`, `lemma61_exp_fcc_21`, `lemma61_prop41_bracket`, `lemma61_tail_fcc`, `lemma61_tail_bcc`, `lemmaA6_tail`, `lemmaA1_rational`. The audit extended the axiom print to *every* declaration of the five modules (138): 127 print [`propext`, `Classical.choice`, `Quot.sound`], two print [`propext`, `Quot.sound`], one prints [`propext`] and eight print no axiom; nothing lies outside the whitelist. A scan of the five sources for `skipKernelTC`, `addDecl`, `native_decide`, `implemented_by`, `sorry` and `axiom` has a single hit, a doc comment in `Shells.lean` saying there is no `sorry` and no `native_decide`; the audit's wider scan (adding `admit`, `unsafe`, `opaque`, `extern`, `csimp`, `macro`, `elab`, `set_option`) adds only four harmless `set_option maxRecDepth 100000 in` lines on the table definitions of `Chains.lean`, an elaboration setting that is not a kernel or trust setting.

5 The threshold κ_* reproduced three ways, and Lemma A.6

Three independent implementations of $G(\kappa) = 2(E_\kappa(L_{\mathrm{B}}) - E_\kappa(L_{\mathrm{F}}))$ (`YBC-C-0001`, [C]). (E) The Ewald split in Arb (python-flint 0.6.0, 320 bits) with closed-form I and $J = e^{-\pi q - c}/(\pi q + c)$, keeping $n \leq 60$, $\ell \leq 240$, with a rigorous geometric tail. (D) A direct real-space sum $\sum e^{-\kappa r}/r$

Tier	Ledger	Statement	Where
[A]	A-0001/2	$N_1..N_{12}$, 17 nonzero M_ℓ ($\ell \le 48$); box count = count over $\mathbb{Z}^3$	Lean
[A]	A-0003/4/5	Lemma A.5 (A.6) $j < 1000$, (A.7) $j < 2000$, (A.8) over $\mathbb{R}$	Lean
[A]	A-0006	constants of the proof of Lemma 6.1 (8 theorems)	Lean
[A]	A-0007	A.6 tail rational $< 7 \times 10^{-21}$; A.1 rational step	Lean
[A]	A-0008	controls: tampered N7, table entry, cell bound rejected; planted `sorry`/axiom caught	Lean
[L]	L-0001/2	Luo–Wei statements, Sections 2–5, taken as stated	paper
[C]	C-0001	κ_* three ways: (E), (Q) agree to 25 digits, (D) contains both; audit: fourth value, 70 digits	Arb, mpmath
[C]	C-0002/3	Lemma A.6 signs (full sums and paper's truncation)	Arb
[C]	C-0004/5	Lemma A.5 re-run; Lemma A.1; Lemma A.3 endpoints	Arb
[C]	C-0007/8	$N_n < 30n$ on $n \le 2100$ only; Lemma 6.1 assembly not formalised	enum., reading
[X]	X-0001/2	IEEE-double direct sum reaches 11 digits (a float64 Ewald reaches about 12); repository guards fail (other lanes)	float, logs

in Arb over about 4 000 shells (integer norms up to 3 800 for FCC and 4 100 for BCC), with a rigorous integral tail from $r_3(k) \le 2(2\sqrt{k}+1)^2$. (Q) The paper's own characterisation (1.4), $\int_1^\infty D(t)\,[t^{-1/2}e^{-c/t} - e^{-ct}]\,dt = 0$, by mpmath quadrature at 40 digits with a secant solver. The shell counts entering (D) and (E) were enumerated here, not copied.

(E) Ewald, Arb	$[1.71792186846990386425087 \pm 1.67 \times 10^{-25}]$
(D) direct sum, Arb	[1.71792186846990386424848 0, 1.71792186846990386425281 7] (radii $\le 4.5 \times 10^{-25}$)
(Q) eq. (1.4), mpmath	1.71792186846990386425087416661
(F) IEEE double (information)	1.7179218684625486 [X]
printed (Thm. 6.1; (A.9))	1.71792186846990; 1.717921868469903 $< \kappa_* <$ 1.717921868469904

(E) and (Q) agree to all 25 printed digits; (D) is limited only by its interval width of about 4×10^{-24} and contains both. All three sit inside the printed enclosure. Our IEEE-double direct sum agrees to 11 digits because of cancellation at $G'(\kappa_*) \approx 1.566 \times 10^{-4}$; 11 digits describes the producer's method, not an intrinsic limit: the audit's float64 Ewald reaches about 12 digits (errors 5×10^{-13} to 4×10^{-12} depending on the splitting parameter), and the intrinsic limit is about $\varepsilon|S|/G'(\kappa_*) \sim 10^{-12}$. Double precision cannot resolve the 10^{-15}-wide enclosure (A.9), so the float control of the preregistration was replaced by (Q).

A fourth value, from the audit ([C]). The independent audit computed κ_* by the classical Yukawa Ewald split with a free splitting parameter η (complementary-error-function real-space kernel, reciprocal-space sum, self term), mpmath at 70 digits, with all lattice constants carried at 70 digits:

$$\kappa_* = 1.71792186846990386425087416660628171879468538.$$

The values for $\eta = 1$ and $\eta = 1.3$ agree to $< 5 \times 10^{-70}$, and the method matches a direct real-space sum at $\kappa = 6$ to $< 1.1 \times 10^{-40}$. It agrees with (E) and (Q) in every printed digit and lies in (D)'s interval and in (A.9); $G'(\kappa_*) = 1.566242612 \times 10^{-4}$. An independent ball-arithmetic run (python-flint 0.9.0, 300

bits, the same Ewald split, rigorous tails) gives $G(1.717921868469903) = [-1.35362654688 \times 10^{-19} \pm 3.3 \times 10^{-31}] < -10^{-19}$ and $G(1.717921868469904) = [2.12616065470 \times 10^{-20} \pm 5 \times 10^{-33}] > 10^{-20}$, so the two signs of Lemma A.6 are reproduced; the controls $\dots 902$ and $\dots 905$ give -2.92×10^{-19} and $+1.78 \times 10^{-19}$.

Lemma A.6 (`YBC-C-0002`, [C]). With full Ewald sums, Arb gives (radius about 10^{-49}):

κ ($\times 10^{-15}$ in the last digits)	$G(\kappa)$
1.717921868469902 (negative control)	-2.91987×10^{-19}
1.717921868469903 (Lemma A.6 lower)	$-1.35363 \times 10^{-19} < -10^{-19}$
1.717921868469904 (Lemma A.6 upper)	$+2.12616 \times 10^{-20} > 10^{-20}$
1.717921868469905 (negative control)	$+1.77886 \times 10^{-19}$
1 / 3 (far from κ_*)	-1.0745×10^{-4} / $+1.5771 \times 10^{-4}$

The direct sum (D) agrees within its balls at every κ. Under the paper's own truncation (FCC $n \le 12$, BCC $\ell \le 48$) plus its printed tail 7×10^{-21} (`YBC-C-0003`), $G_{\mathrm{trunc}} = -1.35362683 \times 10^{-19}$ and $2.12615782 \times 10^{-20}$, so the slacks are 2.84×10^{-20} (lower side) and 4.26×10^{-21} (upper side); the omitted tail is actually 2.83×10^{-26}, five orders below the printed bound. The identity $G =$ eq. (1.4) integral was also checked at both κ (mpmath: $-1.353626547 \times 10^{-19}$, $+2.126160655 \times 10^{-20}$).

Lemma A.5 in Arb (`YBC-C-0004`, [C]) and Lemmas A.1, A.3 (`YBC-C-0005`, [C]). All 1 000 + 2 000 cells hold with no failure; the minimum slacks are 7.52×10^{-4} ((A.6), $j = 999$) and 7.65×10^{-5} ((A.7), $j = 0$), and (A.8) has slack 0.143. Lemma A.1: $P(L_{\mathrm{F}}, 0) = 0.0558589\ldots < 0.055995 < 7/125 = 0.056 < I(16/25, 0) = 0.0561634\ldots$. Printed counts add up: $\sum n_j = 84\,043$ and $1118 + 193 + 86 + 5828 = 7225$.

The Lemma A.3 endpoints are outward-rounded, not a near-tie (`YBC-C-0006`). In Lemma A.3 (A.4), three exponential bounds hold by about 2×10^{-16}: $e^{\alpha_1} = 0.44 - 2.05 \times 10^{-16} \le 11/25$, $e^{\alpha_2} = 0.147 - 1.37 \times 10^{-16} \le 147/1000$, $e^{\beta} = 4/3 + 1.87 \times 10^{-16} \ge 4/3$. The endpoints are outward-rounded: their numerators lie 4.2, 1.05 and 2.5 units of the dyadic denominator beyond the exact logarithms, in the safe direction. For α_1 and α_2 the rationals 11/25 and 147/1000 are themselves intermediate (the proof of Proposition 3.1 has slack of about 3×10^{-3} and 5×10^{-4}), so only the bound for β is tight. These are the tightest margins we met in the appendix; at 320 bits all three hold, and we do not call them an error.

6 Controls

Positive controls (all met). κ_* by three independent implementations (above); the printed digits and the (A.9) enclosure are reproduced; shell counts re-enumerated by brute force up to $k = 4200$ and cross-checked against the r_3 shortcut; the mirror of every Lean check in exact `Fraction` arithmetic reproduces the Arb slacks.

Negative controls (all rejected as intended).

- $\kappa = 1.717921868469902$ and $\dots 905$ give the expected signs ($G < 0$, $G > 0$).
- **Planted wrong multiplicity** $N_7 = 46$ (the true value is 48), `RejectN7.lean`: fails to compile, rc 1, with a kernel error (*application type mismatch*).

- **Tampered table entry**, `C1[500]` multiplied by 1.01 (`RejectChain.lean`): rc 1. **Tampered cell bound**, the (A.7) margin raised to 1/1000 (`RejectCell.lean`): rc 1. For these two the error text is the elaborator's "`decide` failed, reduction got stuck at the `Decidable` instance", which by itself would not distinguish "false" from "stuck". The decisive evidence is `RejectWitness.lean` (rc 0): the theorems `cellA7bad_0_false : cellA7bad 0 = false`, `bad_chain_false : chainLo s1 C1bad = false`, `N7_is_48 : Nc 7 = 48` and `N7_ne_46` are kernel-checked, so the kernel evaluates the tampered checks to `false` and N_7 to 48 (`YBC-A-0008`, [A]). The independent audit reran all of these controls with the same outcomes (rc 1, 1, 1, 0), diffed the tampered definitions against the `Reject*` files (textually identical bodies) and agreed that `RejectWitness` is decisive.
- **Planted `sorry` and planted axiom** (`Controls.lean`): the file compiles with rc 0, which a compile exit code alone would accept; `#print axioms` reports `[propext, sorryAx, Classical.choice, Quot.sound]` for `ctl_sorry` and `[propext, Classical.choice, Quot.sound, ctl_cheat]` for `ctl_axiom`. Both are caught by the axiom audit.
- **Tampered rectangle** (Lemma A.4): not applicable, since no partition data exists to tamper with.

7 What remains open

- **Lemma 6.1 is not in Lean** (`YBC-C-0008`, [C]). Its finite inputs are [A]; the assembly needs Θ as a `tsum` over $\mathbb{Z}^3$, three-dimensional Poisson summation for $D(1) = 0$ with $L_{\mathrm{B}} = L_{\mathrm{F}}^*$, the argument on cells for D' and D, and $N_n < 30n$, $M_\ell < 30\ell$ for all n, ℓ. We read the paper's proof and checked $N_n < 30n$ ($n \le 2100$) and $M_\ell < 30\ell$ ($\ell \le 4200$) by enumeration only (`YBC-C-0007`). We did not open a second kernel budget for this; it is a separate, large lift.
- **Lemma A.4 and Lemma A.2** were not re-run ([C] aim M3c). Lemma A.4 is a finite computation on a dyadic partition of 7 225 rectangles, of which 5 828 need the (a)/(b)/(c) alternatives. The paper itself (p. 4: "the complete interval computations and exact partitions accompany the manuscript") says the partition data exist, but the arXiv abstract page (checked by the audit on 2026-10-27) shows v1 only and no ancillary-file section, and rebuilding Section 5 was outside the budget. We intend to ask the authors for the data; that is the route to M3c. This is the largest unchecked part of the computer-assisted proof.
- **Lemma A.6 is not in Lean**: no `erfc` in the pinned Mathlib. A kernel proof would need either that theory or thousands of certified $\sqrt{\cdot}$ and exp enclosures at 10^{-21}.
- The analytic reductions of Sections 2–5, Theorems 5.1 and 6.1, and the uniqueness of κ_* are taken as stated ([L], `YBC-L-0002`).
- The independent clean recompile (producer $\neq$ verifier) has been done by the AI audit (rc 0, byte-identical oleans). Still owed: a *human* audit of statement adequacy (whether the Lean statements mean what their glosses say), and a statement lock.

8 Claim ledger

`ledger/build_ledger.py` files 21 claims: 8 at Tier A (`YBC-A-0001..0008`), 2 at L, 9 at C (`YBC-C-0001..0009`, the last being the headline) and 2 at X. Each has a content-addressed evidence blob in `ledger/evidence/`. The Elenchus gate (`tools/ledger.py ledger.json --evidence-dir evidence`; verbatim in `ledger/gate_output.txt`) exits with code **1**: **0 blocking findings**

and 8 flags (`LEDGER_UNAUDITED_TIER_A`, one per Tier A row: no *person* has certified that the Lean statements mean what their glosses say). Exit code 1 is the honest state at writing, not a failure to be cleared by editing. An independent AI audit exists (`AUDIT.md`; verdict ACCEPT_WITH_CORRECTIONS; no blocking item); it reran the gate and obtained the identical result (exit 1, 0 blocking, 8 flags). The flags are kept because they correctly say that no *person* has certified the glosses; the ledger rows' `audit` field records the AI audit, not a human one. There is no human audit and no statement lock.

9 Limitations

- **Partial.** See Section 7. We do not claim that Theorem 1.1, 5.1 or 6.1 is verified. Tier A certifies Lean statements about the stated model quantities (unit-covolume shell sums with `Real.pi`, `Real.exp`), nothing more.
- **Deviations from the preregistration.** (1) A.6 is Tier C for a reason other than the kill text gives (no `erfc`). (2) Lemma 6.1 is not kernel-checked, so even the fallback "kernel-check Lemma 6.1 only" was not fully met. (3) The two omitted-tail series are bounded on every partial sum; the monotone limit is not in Lean. (4) The producer's IEEE-double direct-sum control reaches 11 digits (a float64 Ewald reaches about 12), too few for the 13-digit printed value, so the 13-digit float control became mpmath at 40 digits. (5) M3c was blocked, so the tampered-rectangle control is N/A.
- **Compute and process.** The Lean runs were on core 6, which another agent's process also used. Two earlier monolithic compiles of `LemmaA5` were killed with signal 9, unattributed, probably memory pressure from about four 5.7 GB Lean processes of other lanes; the file was then split into four modules with 100-cell blocks. The emitter was re-run once after the oleans were built, to regenerate `Lemma61` only (it is deterministic); `Statements.lean` prints the olean types of the five main theorems and they match the sources. `m3_arb.py` was re-annotated after its first run and re-run with identical key numbers.
- **Verifier status.** The producer's `leanchecker` replay is not independent; the audit's is (private olean directory, the producer's shared olean directory removed from the library path, oleans byte-identical to the producer's, rc 0 on all five modules). The audit is an AI audit; no human has audited the paper or the Lean statements, and no statement lock was registered.
- **Repository guards fail, for pre-existing reasons in other lanes' files.** `antigravity_guard.py` rc 1 (103 findings, hallucinated-import findings in other lanes); `test_rigor_guard.py` rc 1 (tests with no assertions in other modules); `pytest tests/` rc 2 (28 collection errors; no test ran). None lies in this lane's files; this lane added no tests. We report the outputs as they came (`data/logs/guard_*.txt`, `pytest_tests.txt`) and do not call the repository gate passed.
- Interpreter and shell lapses of the producer: the Lean wrapper and guards ran under system `python3` without `-I`; a few wait loops were used in background shells. Neither touched git.
- **The paper text is not redistributed.** The 27-page text extraction used for reading is not included here, since we did not establish its licence; the paper is cited.

10 Reproducibility

Repository AutoevolveAI, branch `worktree-openai-math-discovery`; HEAD when this paper was written: `e5f38b289314c730ed4960104c91a2902b88f8ef`. Nothing was pushed or published by this paper's lane. Copies in `data/`: `code/` (scripts), `lean/` (the twelve Lean sources), `logs/`, `arb_results.json`, `extra_results.json`, `float_results.json`, `shells_k4200.json`, `RESULT.md`, `result.json` (sha256 of all inputs). Lean compile times on core 6: `Basic` 17.8 s, `Shells` 657 s, `Chains` 162 s, `LemmaA5` 2009 s, `Lemma61` 23 s; `leanchecker`: 112, 278, 131, 1601 and 85 s, all rc 0, empty output (producer). The independent audit's `leanchecker` replays on its privately built oleans: 59.3, 168.1, 89.7, 1091.2 and 55.7 s, all rc 0, empty output (`data/leanchecker_replay_audit.json`). Commands (repository root):

```
# every script takes positional arguments; /tmp/m3 must exist; code dir = papers/p4_yukawa_bcc_fcc_check/
    data/code
.venv/bin/python -I papers/p4_yukawa_bcc_fcc_check/data/code/m3_shells.py 4200 /tmp/m3/shells.json
.venv/bin/python -I papers/p4_yukawa_bcc_fcc_check/data/code/with_flint.py .scratchpad/b2_pylib papers/
    p4_yukawa_bcc_fcc_check/data/code/m3_arb.py /tmp/m3/shells.json /tmp/m3/arb.json
.venv/bin/python -I papers/p4_yukawa_bcc_fcc_check/data/code/m3_float.py /tmp/m3/shells.json /tmp/m3/float.
    json
.venv/bin/python -I papers/p4_yukawa_bcc_fcc_check/data/code/with_flint.py .scratchpad/b2_pylib papers/
    p4_yukawa_bcc_fcc_check/data/code/m3_extra.py /tmp/m3/shells.json /tmp/m3/extra.json
.venv/bin/python -I papers/p4_yukawa_bcc_fcc_check/data/code/emit_lemmaA5.py /tmp/m3/lean_emit     # OUT is
    a directory
python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/YukawaBccFcc/Basic.lean --
    olean Programmes.YukawaBccFcc.Basic
                                          # then Shells, Chains, LemmaA5, Lemma61, Axioms (same form)
python -I papers/p4_yukawa_bcc_fcc_check/ledger/build_ledger.py ; python -I papers/p4_yukawa_bcc_fcc_check/
    ledger/run_gate.py  # exit 1 = 0 blocking, 8 flags
```

Run times: shells 0.3 s, Arb 4 s, extra within the two-minute tool limit (not timed separately), mpmath (`m3_float.py`) several minutes, emitter seconds. `m3_arb.py` and `m3_extra.py` need python-flint, which the repository venv does not contain. Under `python -I` neither `PYTHONPATH` nor the user site is read, so the launcher `with_flint.py` (added in this revision; it only extends `sys.path`) takes the directory holding the `flint` package as its first argument. The producer used python-flint 0.6.0 from `.scratchpad/b2_pylib` (a local, untracked directory); the audit used python-flint 0.9.0 from `~/.local/lib/python3.10/site-packages` with the system `python3 -I` (nothing was installed). The path hard-wired inside `m3_arb.py` (`parents[5]/.scratchpad/b2_pylib`) is right only from the original `results/proposal4/2026-10-27/M3/code/` and is ignored here; the scripts were left unmodified so that the hashes in `data/result.json` remain valid. In this revision the shell, Arb, mpmath, extra and emitter commands (not the Lean compiles, which were not rerun) were tested from a scratch copy of the code directory with the repository venv, and the Arb run also with the system `python3 -I` and python-flint 0.9.0 (flint directory `~/.local/lib/python3.10/site-packages`). sha256 of the Lean sources (`data/lean/`) are in `ledger/evidence/` and `data/result.json`. Toolchain: Lean 4.34.1, Mathlib `d13f23b7`, python-flint 0.6.0 (producer) and 0.9.0 (audit), mpmath.

References

[1] S. Luo, J. Wei, *On exact solutions to three-dimensional Yukawa lattice model and BCC to FCC phase transitions*, arXiv:2610.31456v1 [math.AP], 22 Oct 2026. The source of every statement checked here ([L]).

[2] X. Callens, *No connected graph with at most ten vertices is an unweighted death-Birth amplifier of weak selection* (papers/p4_amplifier_census), own preprint in this programme, format and ledger model; not peer reviewed.

[3] X. Callens, *Triangular lattice minimises the planar Yukawa energy*, doi:10.5281/zenodo.23646105 (2026), own preprint; the planar (two-dimensional) Yukawa lattice setting, with the same tier conventions. Not peer reviewed. The present note concerns the three-dimensional problem and does not extend that one.

[4] The mathlib Community, *The Lean mathematical library*, arXiv:1910.09336 (2019); cited for the library only.

[5] F. Johansson, *Arb: efficient arbitrary-precision midpoint-radius interval arithmetic*, IEEE Trans. Comput. 66 (2017) 1281–1292; used through python-flint 0.6.0.